

The magnification-PDF emulator: an equation-level pipeline note

gw-wl-emulator/production_model

September 4, 2026

Verified in this session against gw-wl-emulator commit fcd795b (2026-09-04), by reading the shipped production_model/ package directly (not the exploratory gw-wl-emulator-ar research tree) and by loading the two frozen checkpoints to confirm their recorded hyperparameters. Numbers quoted below come from files under production_model/evaluation/results/, production_model/model/calibr and production_model/PROVENANCE.md in that same commit. A later retrain or a different checkpoint pairing will not automatically match this note.

Abstract

This note documents, equation by equation, what the shipped magnification-PDF emulator (production_model/) actually computes, and why, for a reader who is not an ML specialist. The paper’s Sec. III (“Machine learning”) states the model in prose; this note walks the same object through the actual code, stage by stage, so that every symbol in the paper can be traced to a specific file and line. The object this whole pipeline exists to produce is a single number: $\log p(\ln \mu \mid z_s, \Theta)$, the conditional log-density of the weak-lensing magnification, evaluated in milliseconds instead of requiring a fresh Monte Carlo run of the C++ engine documented in the paper’s Sec. II.

Contents

1	What problem this solves, and the shape of the answer	2
2	Training data and the seven-number context	3
2.1	What “training data” means here	3
2.2	The bundled dataset	3
2.3	The calibrated domain	3
3	Stage 1: a context-dependent coordinate change	4
3.1	Why raw $\ln \mu$ does not work	4
3.2	The coordinate	4
4	Stage 2: normalizing-flow bodies	5
4.1	What a normalizing flow is, briefly	5
4.2	The transform: sum-of-squares polynomial flows (SOSPF)	5
4.3	Body 1: the exact-lower-boundary flow	6
4.4	Body 2: the high-redshift flank correction	7
4.5	Blending the two bodies	7

5	Stage 3: the high-magnification tail	8
5.1	Why the tail is not learned	8
5.2	Peaks-over-threshold regression	8
6	Stage 4: the composite density	8
6.1	Splicing body and tail	8
6.2	Normalization	9
6.3	The flux calibration	9
6.4	The final object	10
7	Held-out accuracy	10
7.1	Width-relative KL divergence, KL_y	10
7.2	Probability integral transform (PIT)	11
7.3	Known, disclosed limitations	11
8	Summary of the pipeline as a chain of interfaces	11
9	References	11

1 What problem this solves, and the shape of the answer

The physics engine (`cpp/lensing.cpp` and friends, documented separately in the paper’s Sec. II) produces samples of the magnification μ by ray-tracing through a simulated matter field. Given a source redshift z_s and six cosmological parameters $\Theta = (h, \Omega_M, \sigma_8, \Omega_B, n_s, z_{\text{eq}})$, it can generate as many Monte Carlo rays as you like, but each ray costs real compute, and a cosmological parameter scan (an MCMC chain, say) needs the density evaluated at thousands of different (z_s, Θ) points. Running the simulator fresh at every point is not feasible.

The emulator’s job is therefore to be a fast stand-in function

$$(\ln \mu, z_s, \Theta) \longmapsto \log p_I(\ln \mu \mid z_s, \Theta)$$

that agrees with what the simulator *would have produced*, everywhere inside a calibrated domain, without requiring new ray-tracing. “Emulator” here does not mean a regression that predicts one or two summary numbers (like a mean or a variance) — it means a full, continuous, normalized probability density over $\ln \mu$ at any queried context. This matters downstream: a likelihood needs the whole shape, not just a couple of moments.

The code is organized as a short pipeline of independent stages, each implemented in its own file under `production_model/model/`, chained together by `model/api.py`:

1. **Standardize the coordinate** (`scale.py`) — because the width of the distribution varies enormously with context, everything downstream works in a rescaled coordinate, not raw $\ln \mu$.
2. **Two learned “bodies”** (`boundary_body.py`, `hybrid_body.py`) — neural density models (normalizing flows) that cover the well-sampled bulk of the distribution.
3. **A calibrated tail** (`tail.py`) — a hand-fit physical model for the rare high-magnification events the flows cannot learn from too few examples.
4. **The composite** (`composite.py`) — splices body and tail into one density, renormalizes it, and applies a small physically motivated correction.

Section 7 closes with how this final object is checked against held-out simulator data.

2 Training data and the seven-number context

2.1 What “training data” means here

Unlike an image classifier, this model is not trained on labelled examples of (input, correct output) pairs — a probability density has no single “correct value” to regress against. Instead, the flow (defined in Sec. 4) is trained by *maximum likelihood*: for every context (z_s, Θ) in the training set, the simulator has already produced a large batch of Monte Carlo magnification rays sampled from the *true* conditional density; training adjusts the flow’s internal parameters so that those observed rays become as probable as possible under the flow’s own predicted density. A density that matches the data well assigns those rays high probability; a bad density assigns them low probability. This is the standard way to fit a density model when you have samples rather than labels.

2.2 The bundled dataset

Per `production_model/data/README.md` and `production_model/PROVENANCE.md` (verified against the same files in this session):

- A base run of **2,055 training / 972 validation / 973 untouched test** configurations, containing 41,006,144 / 19,389,180 / 19,406,954 magnification rays respectively. “Test” here means a checkerboard split held out from every stage of fitting — the number quoted for generalization at the end of this note comes only from that set.
- Two **low-structure augmentation** runs adding 718 more training configurations (14,359,792 rays), added because the narrowest, most-peaked low-redshift/low-structure distributions were under-resolved by the base sample alone.
- A **high-structure, high-redshift augmentation** of 240 training configurations (4,664,309 rays, plus 48 heldout configurations not used in fitting anything), added because the base sample under-resolves the lower flank of the broadest, most extreme distributions.

Each ray is drawn under `PRODUCTION_CONFIG` — the full paper-default physics (subhalo model 5, the σ_8 top-hat anchor, the correlated bias field, filament bias, etc.; see the paper’s Sec. II and `CLAUDE.md` for that pinning) — so the emulator is trained on exactly the physics the paper’s Monte Carlo sections describe, not some cheaper placeholder.

2.3 The calibrated domain

The seven numbers that specify a context are, in the code’s own fixed ordering (`model/api.py`, docstring of `MagnificationPDF`):

$$\mathbf{c} = (z_s, h, \Omega_M, \sigma_8, \Omega_B, n_s, z_{\text{eq}}/1000).$$

Dividing z_{eq} by 1000 is purely a numerical-conditioning choice (it keeps all seven numbers on comparable scales for the neural network) — it carries no physics. Table 1 gives the box actually covered by the bundled data (`production_model/data/README.md`); predictions outside it are unvalidated extrapolations and are excluded from every accuracy number quoted later.

input	minimum	maximum
z_s	0.2	11.993
h	0.55	0.800
Ω_M	0.15	0.500
σ_8	0.40	1.500
Ω_B	0.030	0.070
n_s	0.940	0.990
$z_{\text{eq}}/1000$	3.300	3.500

Table 1: The calibrated domain, from `production_model/data/README.md`.

3 Stage 1: a context-dependent coordinate change

3.1 Why raw $\ln \mu$ does not work

The single largest lever in this whole project, and the reason the pipeline has this stage at all, is a plain empirical fact: the *width* of $p(\ln \mu \mid \mathbf{c})$ — how spread-out the distribution is — changes by roughly a factor of 200 across the calibrated domain. A low-redshift, low-structure context gives an extremely narrow, sharply peaked distribution; a high-redshift, high-structure context gives a very broad one. If you standardized $\ln \mu$ with one single mean and one single spread for the whole training set (the naive approach), the narrow distributions would collapse into a sliver a fraction of a percent wide on that shared axis — the neural network would simply never see them resolved, no matter how much data it was given. The fix is to make the standardization itself a function of context.

3.2 The coordinate

Define

$$y = \frac{\ln \mu - m(\mathbf{c})}{s(\mathbf{c})}, \quad (1)$$

where $m(\mathbf{c})$ (a robust location, like a median) and $s(\mathbf{c})$ (a robust scale, like half the 68% interval) are smooth, closed-form regressions fit once, ahead of any neural-network training, directly on the Monte Carlo samples described in Sec. 2. Every subsequent stage of the pipeline — the flows, the edge location, the tail — is expressed in terms of y , not raw $\ln \mu$, precisely so that one trained model can resolve both the narrowest and the broadest distributions in the domain at once.

Implementation. `model/scale.py::loc_scale` computes

$$\log s(\mathbf{c}) = \mathbf{x}(\mathbf{c})^\top \boldsymbol{\beta}_{\log s}, \quad (2)$$

$$m(\mathbf{c}) = s(\mathbf{c}) \cdot \mathbf{x}(\mathbf{c})^\top \boldsymbol{\beta}_m, \quad (3)$$

where $\mathbf{x}(\mathbf{c})$ is a fixed, hand-chosen feature vector (not learned by gradient descent — it is a linear ridge fit) built by `model/features.py::scale_features`: a degree-4 polynomial in $\ln z_s$, plus $\ln \sigma_8, \ln \Omega_M, \ln h, \Omega_B, n_s, z_{\text{eq}}$, plus cross terms with $\ln z_s$ and $\ln z_s^2$ (19 features total, standardized by their own training-set mean/std before the dot product). The coefficients $\boldsymbol{\beta}_{\log s}, \boldsymbol{\beta}_m$ are stored in the checkpoint (`stats["scale_fit"]`) — they are frozen numbers baked in at fit time, not something the flow adjusts during its own training.

Why this functional form. $\ln s$, not s itself, is what is regressed, guaranteeing $s > 0$ for any input (an exponential of anything is positive) without a constrained optimizer. m is fit as s times a

separate regression rather than independently, which keeps the location estimate naturally tied to the local width rather than letting the two drift apart in an unphysical way. The specific basis — a low-order polynomial in $\log z_s$ together with logs of the amplitude-like parameters σ_8, Ω_M, h — was chosen empirically because $\sigma(\ln \mu)$ is known (from the analytic theory referenced in the paper’s Sec. II and CLAUDE.md’s `sigma_full_analytic_note.md`) to grow like $z_s^{3/2}$ at low redshift and to be roughly a power of σ_8 at fixed redshift; a basis built from those variables’ logs fits this shape far better (measured: $\sim 3\%$ held-out relative error, flat across redshift) than the same regression built on the low-redshift-only edge feature basis of Sec. 4 ($\sim 15\%$, and worse per band).

Because y is a rescaling of $\ln \mu$, densities transform with a Jacobian: if $p_y(y | \mathbf{c})$ is the density the flow predicts in y -space, the density in $\ln \mu$ -space is

$$p_x(\ln \mu | \mathbf{c}) = \frac{1}{s(\mathbf{c})} p_y(y | \mathbf{c}), \quad (4)$$

and, converting once more from $\ln \mu$ to μ itself,

$$p_I(\mu | \mathbf{c}) = \frac{p_x(\ln \mu | \mathbf{c})}{\mu}. \quad (5)$$

Both factors of $1/s$ and $1/\mu$ appear explicitly in the code (e.g. `model/boundary_body.py:56`, `model/api.py:30`) — they are not optional; forgetting either would silently misnormalize the density.

4 Stage 2: normalizing-flow bodies

4.1 What a normalizing flow is, briefly

A normalizing flow is a way of building a flexible, trainable probability density out of a simple one. Start with an easy reference density (here, a Normal distribution, or a variant of it) over an internal variable, then pass it through an invertible, differentiable transformation $T_{\mathbf{c}}$ — invertible so that probability is conserved, and depending on the context $\mathbf{c}$ so the shape can change with z_s and cosmology. The change-of-variables formula for densities,

$$p(y) = p_{\text{base}}(T_{\mathbf{c}}^{-1}(y)) \left| \frac{dT_{\mathbf{c}}^{-1}}{dy} \right|, \quad (6)$$

converts a density on the simple reference variable into a density on y . Training a flow means adjusting the parameters of $T_{\mathbf{c}}$ (via a neural network, the “conditioner”) so that Eq. 6 assigns high probability to the training rays. This is the general recipe; the specific transform used here is described next.

4.2 The transform: sum-of-squares polynomial flows (SOSPF)

The transform $T_{\mathbf{c}}$ is built by a *sum-of-squares polynomial flow* [?]: its derivative $dT_{\mathbf{c}}/du$ is constructed as a sum of squares of low-degree polynomials (plus a small positive constant), which is automatically non-negative everywhere, and integrating a non-negative function always gives a monotonically increasing one — so $T_{\mathbf{c}}$ is guaranteed invertible by construction, with no separate constraint needed. Compared to the more common spline-based flows, this gives a smooth, everywhere-infinitely-differentiable transport with no knot points, which matters here because a spliced density (Sec. 6) is much easier to keep smooth if its pieces are already smooth.

Implementation. `model/architecture.py::build_flow` constructs the transform via `zuko.flows.polynomial`. Both shipped checkpoints (loaded and inspected directly in this session) use identical hyperparameters:

- 3 stacked transformations (`transforms=3`);
- each built from 5 polynomials of degree 8 (`polynomials=5, degree=8`);
- a conditioner network with 2 hidden layers of 128 units each, reading in the full 7-number context $\mathbf{c}$;
- **70,412 trainable parameters** per body, confirmed against `docs/MODEL_CARD.md`.

Training uses the Adam optimizer directly on the negative log-likelihood of the training rays (maximum likelihood, as described in Sec. 2).

4.3 Body 1: the exact-lower-boundary flow

The physical magnification distribution has a genuine hard edge at low μ — a lensing “empty-beam” cutoff below which the density is essentially zero, not merely small. A generic flow does not reproduce a hard edge naturally; it will smear it out. The fix used here is to make the *coordinate itself* enforce the edge, so the flow never has to learn a discontinuity.

A separate ridge regression (frozen at fit time, same style as Sec. 3) predicts an edge location $y_{\text{edge}}(\mathbf{c})$ and a redshift-only edge uncertainty $\sigma_{\text{edge}}(z_s)$ (an interpolated lookup table in $\log z_s$). The boundary used by the flow is placed a fixed margin below the fitted edge,

$$y_b(\mathbf{c}) = y_{\text{edge}}(\mathbf{c}) - 4\sigma_{\text{edge}}(z_s), \quad (7)$$

(`model/boundary_body.py::controls`, margin $k_{\text{margin}} = 4$ read directly from the checkpoint), and the flow is trained not in y but in

$$u = \ln(y - y_b(\mathbf{c})), \quad y > y_b(\mathbf{c}). \quad (8)$$

Because $u \rightarrow -\infty$ as $y \rightarrow y_b^+$, the log-density under a well-behaved base distribution automatically goes to $-\infty$ there too — the density in y vanishes continuously at the boundary by construction, not because the flow learned to suppress it. Converting the flow’s log-density back to y -space picks up a Jacobian factor $du/dy = 1/(y - y_b)$:

$$\log p_y(y | \mathbf{c}) = \log p_u(u | \mathbf{c}) - \ln(y - y_b(\mathbf{c})), \quad (9)$$

implemented exactly this way at `model/boundary_body.py:56` (`lp_u - u - np.log(s)`, where the extra $-\ln s$ is Eq. 4’s conversion back to $\ln \mu$). Below y_b the code simply returns $-\infty$ (`model/boundary_body.py:45{46}`).

This body — the “exact-support” body — is the default everywhere and supplies the complete model at low redshift. Its only known weakness (next section) is at high redshift, where the boundary-anchored coordinate is too restrictive to also capture a separately-observed asymmetric left flank.

4.4 Body 2: the high-redshift flank correction

At high $z_s (\gtrsim 2)$, the simulated distributions develop a left flank that the symmetric, boundary-anchored Body 1 does not fit well. A *second*, independently trained SOSPF flow is used only to patch this region. It is trained directly in y (no boundary transform), with an asymmetric *split-normal* base distribution — a Normal-like density with two different widths on either side of zero,

$$p_{\text{base}}(y) = \frac{1}{\sigma_L + \sigma_R} \sqrt{\frac{2}{\pi}} \exp \left[-\frac{1}{2} \left(\frac{y}{\sigma_{L \text{ or } R}} \right)^2 \right], \quad \sigma_L = 0.6, \sigma_R = 1.3, \quad (10)$$

(`model/hybrid_body.py::SplitNormal`, with σ_L, σ_R read from the checkpoint) — an asymmetric starting point that gives the flow a head start on a distribution that is itself asymmetric, rather than asking a symmetric base to learn the asymmetry entirely through the transform.

4.5 Blending the two bodies

The two bodies are combined with smooth, compactly-supported “switch” functions rather than a hard if/else, so the composite stays continuous and differentiable. `model/hybrid_body.py::compact_switch(x, lo, hi)` implements the standard smooth bump function

$$w(t) = \begin{cases} 0 & t \leq 0 \\ \left[1 + \exp \left(\frac{1}{t} - \frac{1}{1-t} \right) \right]^{-1} & 0 < t < 1, \\ 1 & t \geq 1 \end{cases} \quad t = \frac{x - \text{lo}}{\text{hi} - \text{lo}}, \quad (11)$$

which is exactly zero for $x \leq \text{lo}$, exactly one for $x \geq \text{hi}$, and infinitely differentiable everywhere including at the two junctions (its derivatives all vanish there) — a standard bump-function construction that avoids introducing any kink where the two pieces meet.

The blend weight (`model/hybrid_body.py::load_hybrid_body`) is

$$w(\mathbf{c}, y) = w_z(z_s) \cdot [1 - w_y(y)], \quad (12)$$

with w_z a switch in $\log z_s$ active over $2 < z_s < 3.5$ (`Z_BLEND_LO, Z_BLEND_HI`) and w_y a switch in y active over $-1 < y < 0$ (`Y_BLEND_LO, Y_BLEND_HI`) — so Body 2 can only contribute at high redshift, and only on the left flank, never touching the core or any low-redshift context. Densities, not log-densities, are mixed,

$$p(y \mid \mathbf{c}) = [1 - w(\mathbf{c}, y)] p_1(y \mid \mathbf{c}) + w(\mathbf{c}, y) p_2(y' \mid \mathbf{c}), \quad (13)$$

which is the only way to combine two densities that keeps the result a valid, positive density (mixing log-densities does not, in general, produce something that integrates to something sensible). $y' = y - s \cdot \Delta$ is a small structure-dependent horizontal shift of Body 2’s argument (`flank_shift`, a function of z_s and of a combined “structure” variable $\sigma_8 \sqrt{\Omega_M / 0.3}$) that fine-tunes where Body 2’s flank lands for the most extreme, high-structure contexts. A third, narrow compact gate (`ISLAND_*` constants) additionally suppresses a weak, unphysical low-density branch that otherwise appears in Body 2’s output for a small subset of high-structure configurations — a cleanup, not part of the physical model.

5 Stage 3: the high-magnification tail

5.1 Why the tail is not learned

Maximum-likelihood training, by construction, fits the density where the data is — and rare high-magnification events are, by definition, rare: a typical training context may contribute only a handful of rays above $\mu \sim 2\text{--}3$ out of tens of thousands. A flow asked to fit that region by likelihood alone has almost no signal to learn from, and produces inconsistent, undertrained tails. This part of the density is instead built directly from physics and from targeted statistics, independent of the flow’s own training loss.

5.2 Peaks-over-threshold regression

For three fixed magnification thresholds $\mu = 2, 3, 8$, the *survival probability* $S(\mu) = P(\mu' > \mu)$ is estimated directly from the simulator’s own Monte Carlo counts, and then *that* quantity — a much better-behaved regression target than a raw tail density — is fit as a smooth function of context by another frozen ridge regression (`model/tail.py::pot_tail_logp`, coefficients in `model/calibration/tail_fit.json`, feature basis `model/features.py::tail_features`: polynomials in $\log z_s$, the same linear cosmology terms as before, plus explicit $1/\sigma_8, 1/\sigma_8^2$ terms to capture the tail’s stronger dependence on the matter-power amplitude).

Between the three calibrated points, the log-slope of the survival curve is interpolated smoothly. Writing $x = \ln \mu$ and $x_2, x_3, x_8 = \ln 2, \ln 3, \ln 8$, define local power-law slopes between adjacent anchors,

$$k_1 = \text{clip}\left(\frac{\ln S_2 - \ln S_3}{x_3 - x_2}, k_{\min}, k_{\max}\right), \quad k_2 = \text{clip}\left(\frac{\ln S_3 - \ln S_8}{x_8 - x_3}, k_{\min}, k_{\max}\right), \quad (14)$$

and blend between them with a soft interpolation built from the softplus function $\text{softplus}(t) = \ln(1 + e^t)$ (`model/tail.py:24{45}`), so that $\ln S(x)$ behaves like slope k_1 just above $\mu = 2$, smoothly bends toward slope k_2 near $\mu = 3$, and beyond $\mu = 8$ approaches the fixed asymptotic slope 1 in $\ln S$ — i.e. $S(\mu) \propto \mu^{-1}$, equivalently

$$p_I(\mu \mid \mathbf{c}) \propto \mu^{-2} \quad (15)$$

at large magnification, since $p = -dS/d\mu$. This is the standard fold-caustic result: the cross-section for magnification above μ falls as μ^{-2} near a fold caustic [?], matching the paper’s Sec. II discussion of the same asymptote. The final tail log-density is obtained by differentiating the interpolated $\ln S(x)$ and adding back $\ln(-dS/dx)$ (`model/tail.py:43{45}`) — this is the standard peaks-over-threshold construction from extreme value theory, here with a smoothly-varying effective index rather than a single fixed Pareto tail index, because the measured slope genuinely changes between $\mu = 2$ and $\mu = 8$. This asymptote is not imposed for $z_s \leq 0.3$ (`model/composite.py::LOW_Z_TAIL_OFF`), where the simulator does not resolve a distinct asymptotic regime at those redshifts.

6 Stage 4: the composite density

6.1 Splicing body and tail

`model/composite.py::make_composite` produces the final callable. For a given context, it first looks for the lowest magnification above $\mu = 2$ at which the body’s and the tail’s log-densities cross (`_crossing`), scanning a fine grid and linearly interpolating the sign change of $\log p_{\text{body}} - \log p_{\text{tail}}$.

If a crossing exists, the two pieces are blended over a fixed window of width 0.3 in $\ln \mu$ starting at the crossing, using the same smooth switch as Eq. 11:

$$p_{\text{raw}}(\mu \mid \mathbf{c}) = [1 - w(x)] p_{\text{body}}(\mu \mid \mathbf{c}) + w(x) p_{\text{tail}}(\mu \mid \mathbf{c}), \quad x = \ln \mu. \quad (16)$$

If *no* crossing is found — meaning the body and tail never sensibly agree on that grid — the code does not silently trust an unconstrained extrapolation of either piece. Instead it falls back to a *fixed*, physics-calibrated window $\mu = 2 \rightarrow 3$ (`model/composite.py:104`, comment: `plan_mode = "crossing_or_pot_2_3"`), and even then only after `_fixed_rescue_window` verifies both endpoints are finite; if they are not, the code raises `UnverifiedFallbackError` rather than return a silently wrong density. This is a deliberate design choice: an emulator feeding a cosmological likelihood must fail loudly rather than quietly extrapolate somewhere it has no support.

A separate diagnostic, `_fallback_diagnostics`, checks body/tail agreement at several reference magnifications (20, 50, 100) and is used at very low redshift ($z_s \leq 0.3$, where the μ^{-2} asymptote is not imposed at all) to decide whether the raw body alone is already safe to use without any tail at all.

6.2 Normalization

After splicing, the density is not guaranteed to integrate to exactly 1 (the two pieces were each independently fit / calibrated). The composite therefore evaluates the unnormalized density on a grid — a coarse grid covering the full domain plus a fine grid centered on the body’s own (m, s) (`_grid`, Eq. 1’s m, s again, this time used to place resolution where the mass actually is) — and divides by the trapezoidal-rule integral:

$$p_I(\mu \mid \mathbf{c}) = \frac{p_{\text{raw}}(\mu \mid \mathbf{c})}{\int p_{\text{raw}}(\mu' \mid \mathbf{c}) d \ln \mu'}. \quad (17)$$

6.3 The flux calibration

A physical sum rule constrains the true magnification distribution: $\langle 1/\mu \rangle = 1$ (flux/photon-number conservation along random lines of sight). The fitted composite need not satisfy this exactly, so a small rigid shift δ in $\ln \mu$ is applied,

$$\delta(\mathbf{c}) = \text{clip}\left(\ln \langle e^{-\ln \mu} \rangle_{\text{model}}, -\delta_{\text{max}}, \delta_{\text{max}}\right), \quad \delta_{\text{max}} = 0.05, \quad (18)$$

(`model/composite.py::context`, `FLUX_DELTA_MAX = 0.05`), and the final density is evaluated at the shifted argument, $p_I(\mu \mid \mathbf{c}) \rightarrow p_I(\mu e^{-\delta} \mid \mathbf{c})$, i.e. `model/composite.py:141`’s `unnormalized(x - delta, ...)`. The clip is a deliberate trust region: an earlier, unclipped version of this correction let a badly-fit tail at one extreme corner of the domain drag the entire peak by half a decade in $\ln \mu$ (a translation this large is not a small correction, it visibly moves the peak off the simulated data); $\delta_{\text{max}} = 0.05$ was chosen because legitimate corrections measured elsewhere in the domain are $\lesssim 0.02$, so the clip essentially never binds on a healthy fit but strictly bounds the damage a bad one can do.

6.4 The final object

Putting Eqs. 5, 16, 17 and 18 together, the quantity every downstream consumer of this emulator actually calls (`model/api.py::MagnificationPDF.pdf_mu`) is

$$p_I(\mu \mid z_s, \Theta) = \frac{1}{\mu} \cdot \frac{[1 - w(x')] p_{\text{body}}(\mu' \mid z_s, \Theta) + w(x') p_{\text{tail}}(\mu' \mid z_s, \Theta)}{\int [\dots] d \ln \mu''}, \quad x' = \ln \mu - \delta, \mu' = e^{x'}.$$
(19)

This is the single function the paper’s Sec. III describes in prose and that a cosmological analysis would call in place of the Monte Carlo engine.

7 Held-out accuracy

Every number in this section is read directly from `production_model/evaluation/results/` in this session (commit `fcd795b`), computed by the scripts described below rather than asserted.

7.1 Width-relative KL divergence, KL_y

Ordinary binning in $\ln \mu$ would be blind at low redshift for the same reason raw $\ln \mu$ standardization failed in Sec. 3: a fixed bin width can hold 99% of a narrow distribution’s mass in one or two cells. `evaluation/evaluate_kl.py` instead bins in the standardized coordinate y of Eq. 1, on a fixed 120-bin grid over $-6 < y < 12$, so a bin’s width in physical $\ln \mu$ automatically scales with that context’s own $s(\mathbf{c})$. For each held-out configuration, the histogram of the simulator’s own rays in y (bins with fewer than 5 rays dropped) is compared against the model’s density integrated over the same bins (12-point trapezoidal quadrature per bin), and

$$KL_y = \sum_i q_i \ln \frac{q_i}{p_i}, \quad (20)$$

with q_i, p_i the (renormalized, on the retained bins) simulated and modeled bin probabilities — the standard Kullback–Leibler divergence, measuring how much extra information is needed to describe the simulator’s histogram if you had only the model’s prediction.

split	configurations	median KL_y
full validation	972	0.002233
standard acceptance sample	243	0.002253
untouched test	973	0.002227

Table 2: From `evaluation/results/{validation,test}_summary.json` and `validation_acceptance_243_summary.json`. The test set was never used to fit anything (body, edge, tail, or scale) — its close agreement with the validation number (within 0.3%) is the evidence against a generalization gap.

On the untouched test set the 90th/99th percentile/maximum KL_y are 0.004534/0.01221/0.03058 (`evaluation/results/test_summary.json`); the worst single case sits at $z_s = 10.43$, $\Omega_M = 0.496$, $\sigma_8 = 1.491$ — a high-redshift, high-structure corner of the domain, consistent with the known-limitations list below.

7.2 Probability integral transform (PIT)

KL_y checks local shape; PIT checks whether the model is a *calibrated* density in the stronger cumulative sense. For every simulator ray $\ln \mu_i$ in a context, its PIT value is $U_i = F_{\text{model}}(\ln \mu_i \mid \mathbf{c})$, the model’s own cumulative distribution function evaluated at that ray. If the model is exactly correct, U_i is uniformly distributed on $[0, 1]$ by construction (a basic property of any CDF applied to its own distribution) — a systematic departure from uniform reveals a systematic mis-calibration the median KL_y could hide. Applied to all $\sim 1.94 \times 10^7$ test rays across all 973 test configurations, pooled, the Kolmogorov–Smirnov distance from uniform is 0.00543 (paper_prod/scripts/compute_pit.v1.py, result recorded in paper_prod/ML_CLAIM_LEDGER_2026-09-03.md §7); it rises to 0.0139 restricted to $z_s \geq 4$, consistent with the residual edge/tail errors concentrating at high redshift.

7.3 Known, disclosed limitations

Per docs/MODEL_CARD.md and PROVENANCE.md (both re-read in this session): of the 243-configuration acceptance sample, 40 configs have their modeled-to-simulated low-tail mass ratio (at the empirical 10^{-3} quantile) outside $[0.5, 2]$, and 21 assign exactly zero modeled mass below the empirical 10^{-4} quantile — i.e. the composite’s window placement (Sec. 6) is occasionally too aggressive at the very edge, a defect concentrated in the extreme tails and invisible to the bulk-weighted median KL_y . Three of 243 have $KL_y > 0.01$.

8 Summary of the pipeline as a chain of interfaces

Table 3 restates the whole pipeline as inputs/outputs, which is the fastest way to see how the pieces compose without re-reading the equations.

stage	implementing code	interface
standardization	model/scale.py	$(z_s, \Theta) \rightarrow (m, s)$
edge location	model/boundary_body.py::controls	$(z_s, \Theta) \rightarrow y_{\text{edge}}, y_b$
Body 1 (exact edge)	model/boundary_body.py	$(\ln \mu, z_s, \Theta) \rightarrow \log p_1$
Body 2 (high- z flank)	model/hybrid_body.py	$(\ln \mu, z_s, \Theta) \rightarrow \log p_2$
blended body	model/hybrid_body.py::load_hybrid_body	$(\ln \mu, z_s, \Theta) \rightarrow \log p_{\text{body}}$
tail	model/tail.py	$(\ln \mu, z_s, \Theta) \rightarrow \log p_{\text{tail}}$
composite	model/composite.py	body + tail $\rightarrow \log p_I(\mu \mid z_s, \Theta)$, normalized, flux-calibrated
public API	model/api.py::MagnificationPDF	the one object downstream code calls

Table 3: The pipeline as a chain of interfaces.

9 References

- P. Jaini, K. A. Selby, Y. Yu, *Sum-of-Squares Polynomial Flow*, ICML 2019 (arXiv:1905.02325) — the SOSPF transform used for both flow bodies (Sec. 4).
- R. D. Blandford, R. Narayan, *Fermat’s principle, caustics, and the classification of gravitational lens images*, ApJ 310, 568 (1986) — source of the μ^{-2} (image-plane) fold-caustic asymptote used in the tail model (Sec. 5).

- `production_model/PROVENANCE.md`, `production_model/docs/MODEL_CARD.md` — the package’s own provenance and model-card records, cross-checked directly against the loaded checkpoints and result files in this session.